

CS516: Parallelization of Programs

Dependency Analysis

Vishwesh Jatala

Assistant Professor

Department of CSE

Indian Institute of Technology Bhilai

vishwesh@iitbhilai.ac.in



Recap

```

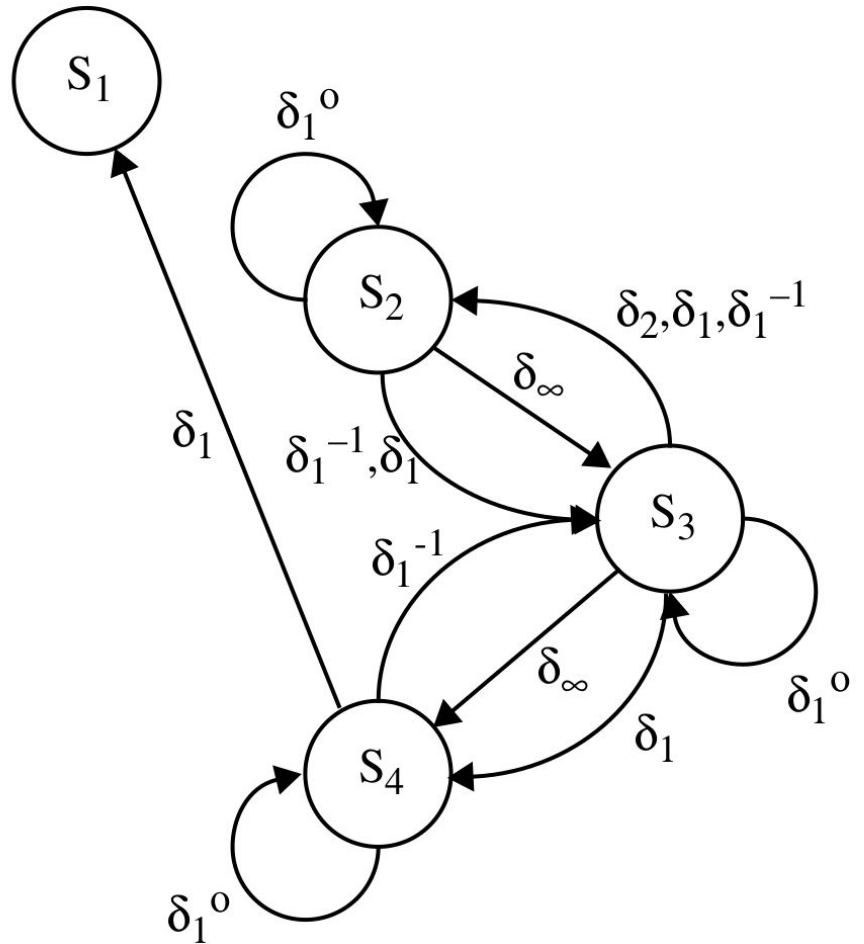
DO I = 1, 100
S1    X(I) = Y(I) + 10
      DO J = 1, 100
S2        B(J) = A(J,N)
          DO K = 1, 100
S3            A(J+1,K) = B(J) + C(J,K)
              ENDDO
S4        Y(I+J) = A(J+1, N)
          ENDDO
      ENDDO

```

	S1	S2	S3	S4
S1	-	-	-	-
S2	-	δ_1^0	$\delta_1 \delta_\infty \delta_1^{-1}$	-
S3	-	$\delta_2 \delta_1 \delta_1^{-1}$	δ_1^0	$\delta_1 \delta_\infty$
S4	δ_1	-	δ_1^{-1}	δ_1^0

Dependency Graph

	S1	S2	S3	S4
S1	-	-	-	-
S2	-	δ_1^0	$\delta_1 \delta_\infty \delta_1^{-1}$	-
S3	-	$\delta_2 \delta_1 \delta_1^{-1}$	δ_1^0	$\delta_1 \delta_\infty$
S4	δ_1	-	δ_1^{-1}	δ_1^0

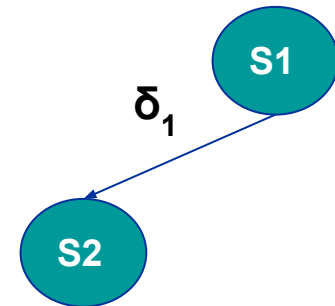


How to Extract Parallelism?

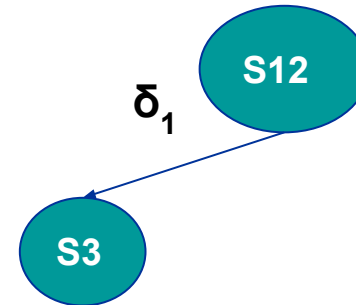
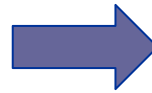
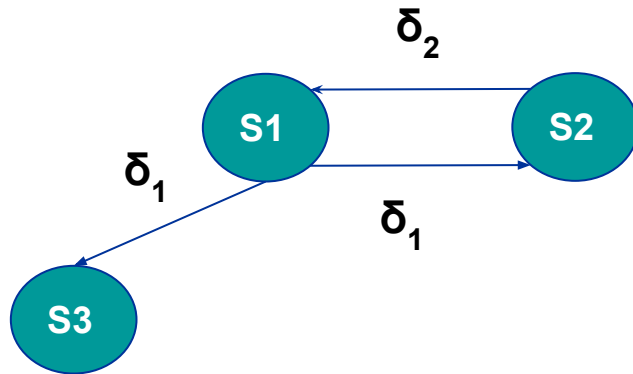
```
      DO I = 1, N  
S1      A(I+1) = B(I) + C  
S2      D(I) = A(I) + E  
      ENDDO
```



```
      PARALLEL DO I = 1, N  
S1      A(I+1) = B(I) + C  
      ENDDO  
  
      PARALLEL DO I = 1, N  
S2      D(I) = A(I) + E  
      ENDDO
```

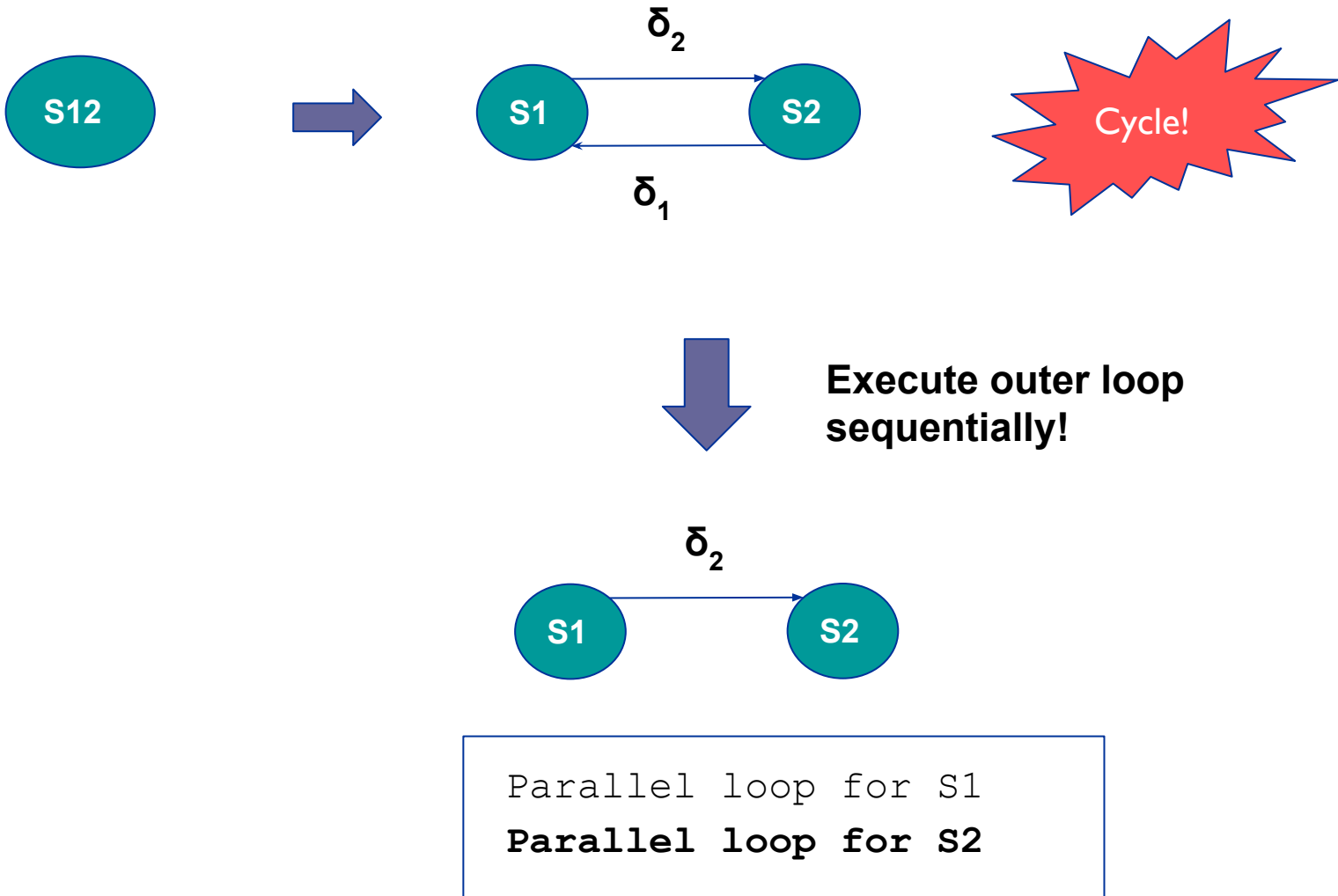


How to Extract Parallelism?



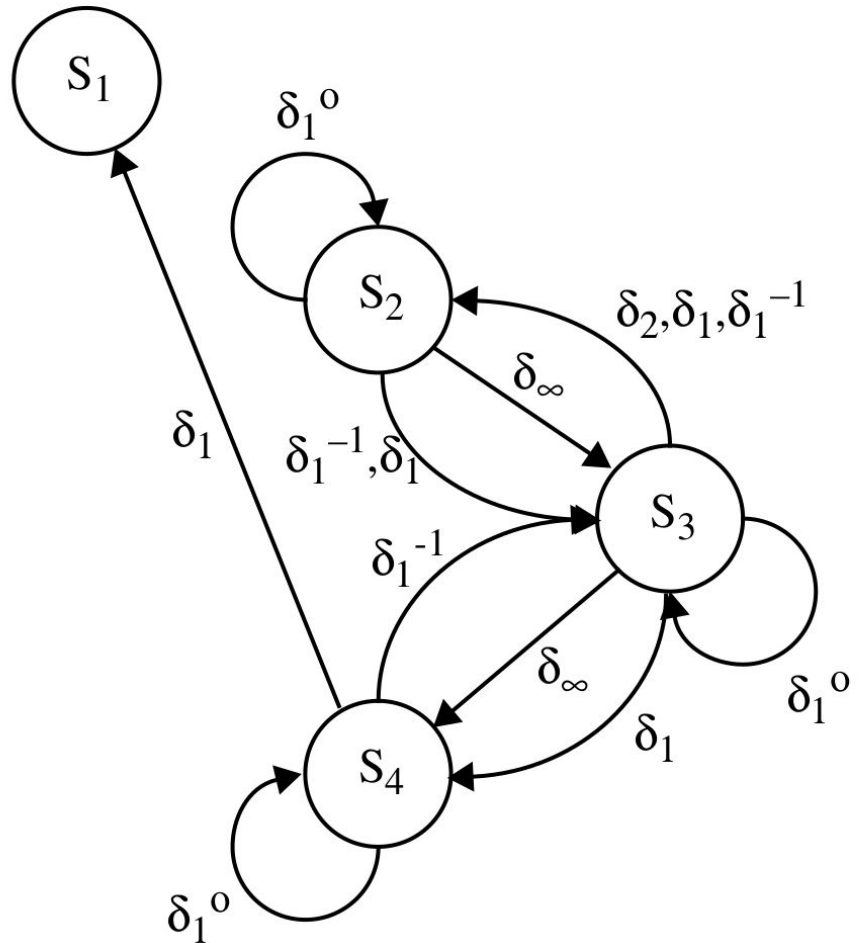
Code for S12
Parallel Loop for S3

How to Extract Parallelism?

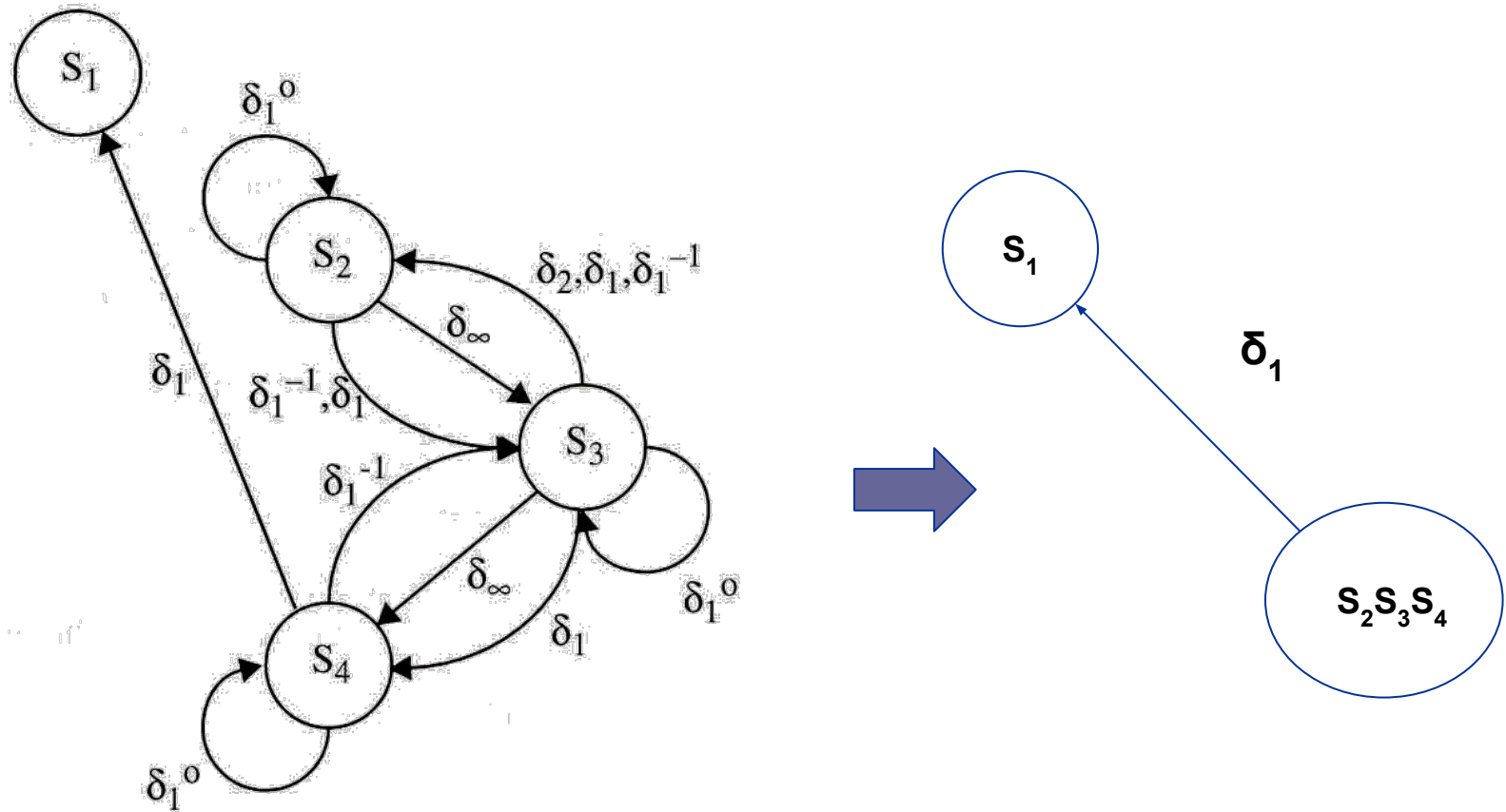


Dependency Graph

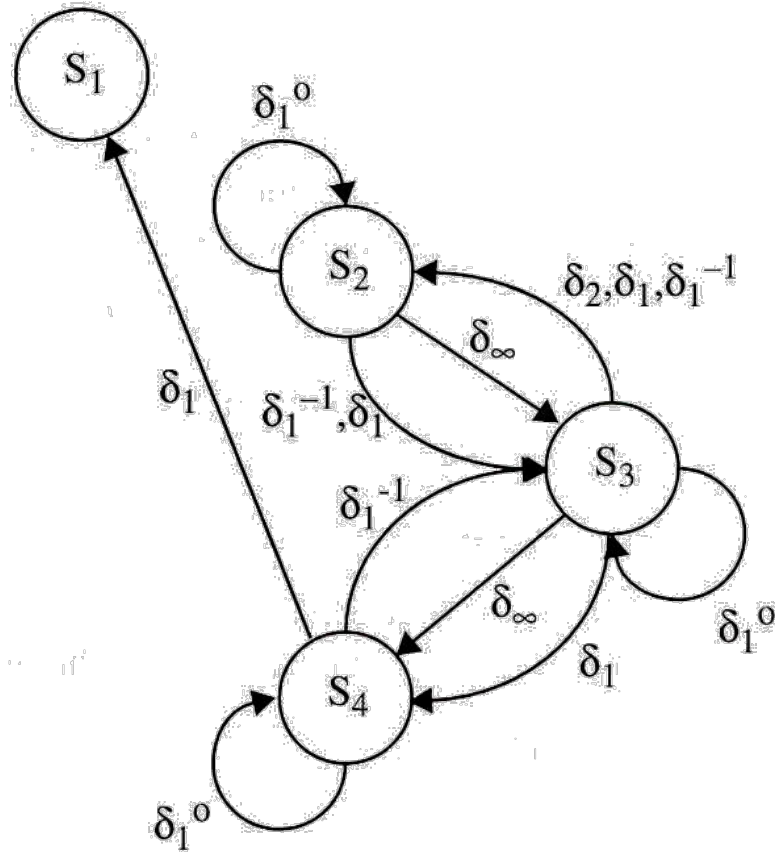
	S1	S2	S3	S4
S1	-	-	-	-
S2	-	δ_1^0	$\delta_1 \delta_\infty \delta_1^{-1}$	-
S3	-	$\delta_2 \delta_1 \delta_1^{-1}$	δ_1^0	$\delta_1 \delta_\infty$
S4	δ_1	-	δ_1^{-1}	δ_1^0



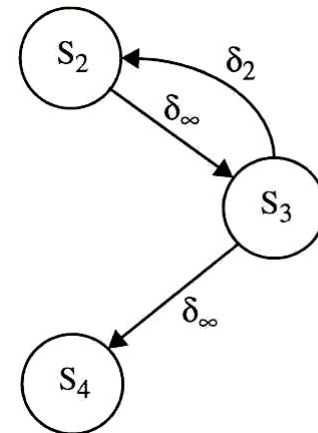
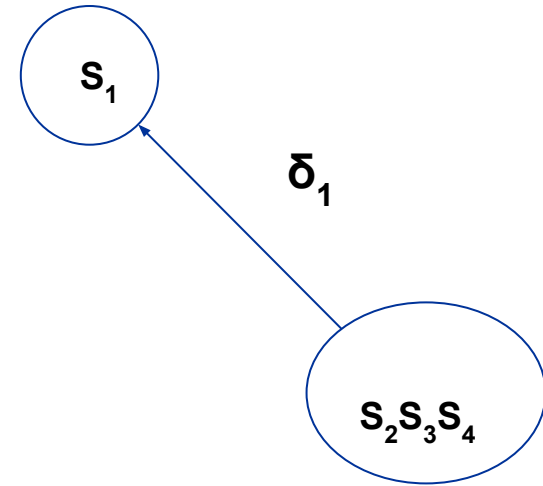
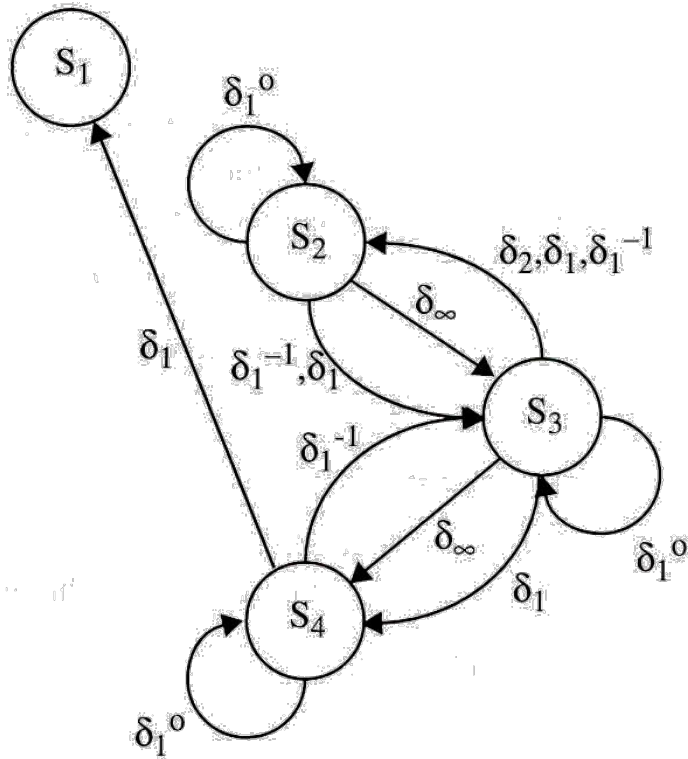
Dependency Graph



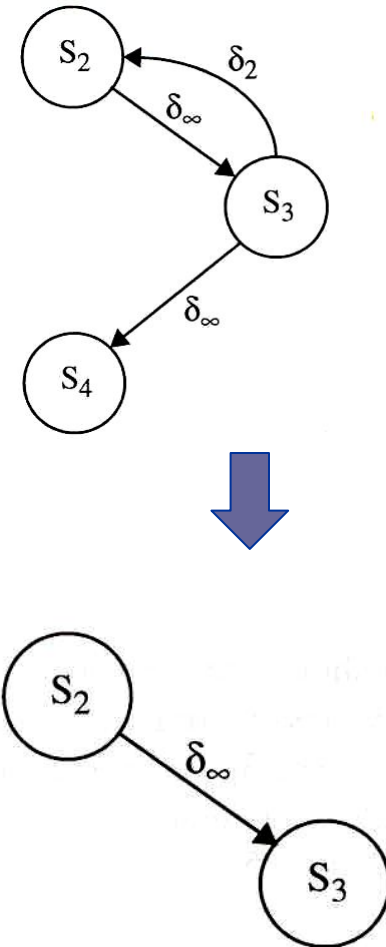
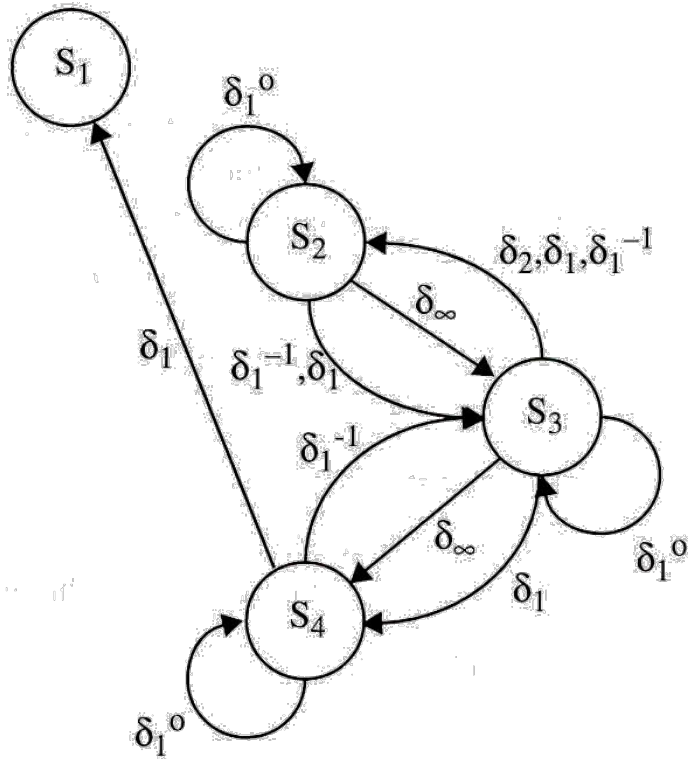
Dependency Graph



Dependency Graph



Dependency Graph

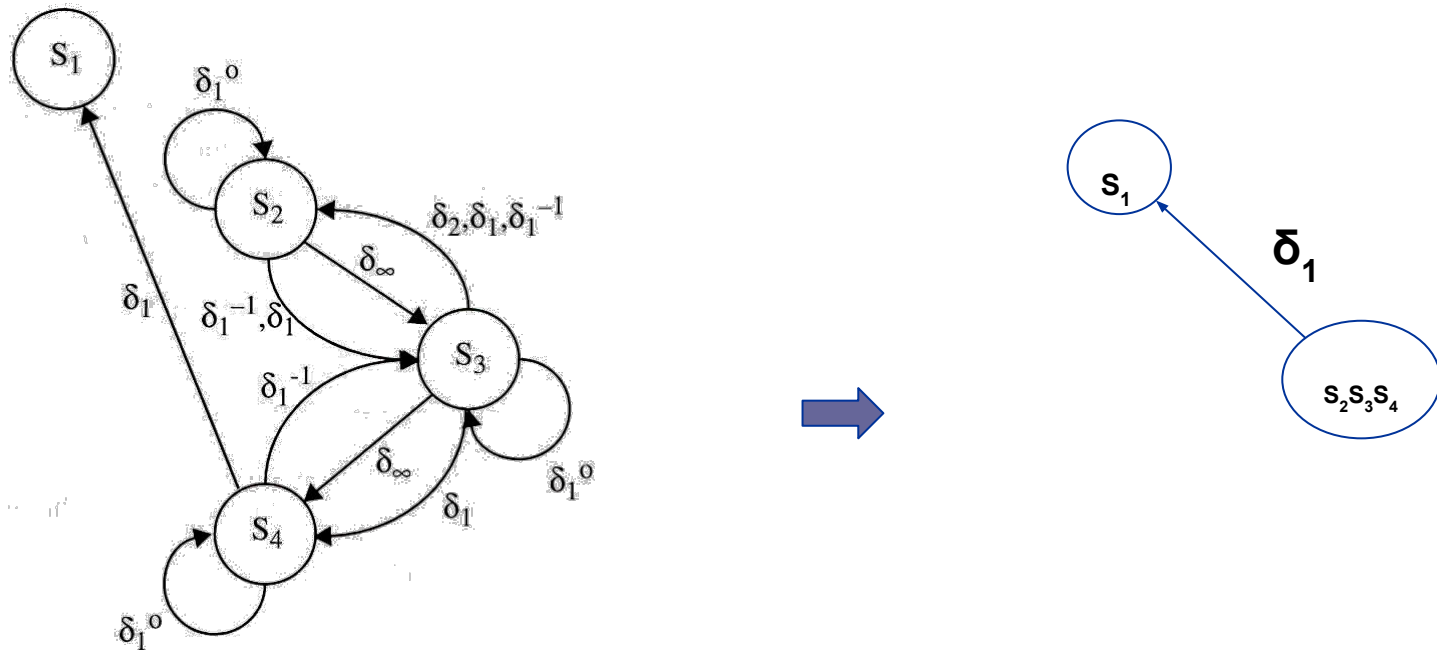


Algorithm

```
procedure codegen(R, k, D);  
    // R is the region for which we must generate code.  
    // k is the minimum nesting level of possible parallel loops.  
    // D is the dependence graph among statements in R..  
    find the set {S1, S2, ... , Sm} of maximal strongly-connected  
        regions in the dependence graph D restricted to R;  
    construct Rp from R by reducing each Si to a single node and  
        compute Dp, the dependence graph naturally induced on Rp by D;  
    let {p1, p2, ... , pm} be the m nodes of Rp numbered in an order  
        consistent with Dp (use topological sort to do the numbering);  
    for i = 1 to m do begin  
        if pi is cyclic then begin  
            generate a level-k DO statement;  
            let Di be the dependence graph consisting of all dependence edges in D  
                that are at level k+1 or greater and are internal to pi;  
            codegen (pi, k+1, Di);  
            generate the level-k ENDDO statement;  
        end  
        else  
            generate a parallel loop for pi at level k.  
        end  
    end
```

Example

- codegen called at the outermost level



Example

- S1 will be parallelized

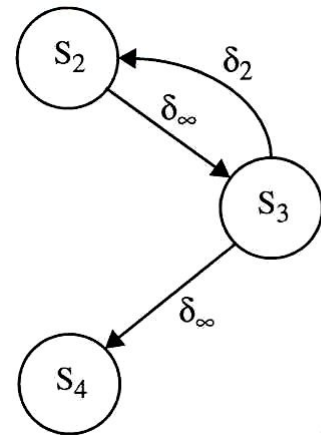
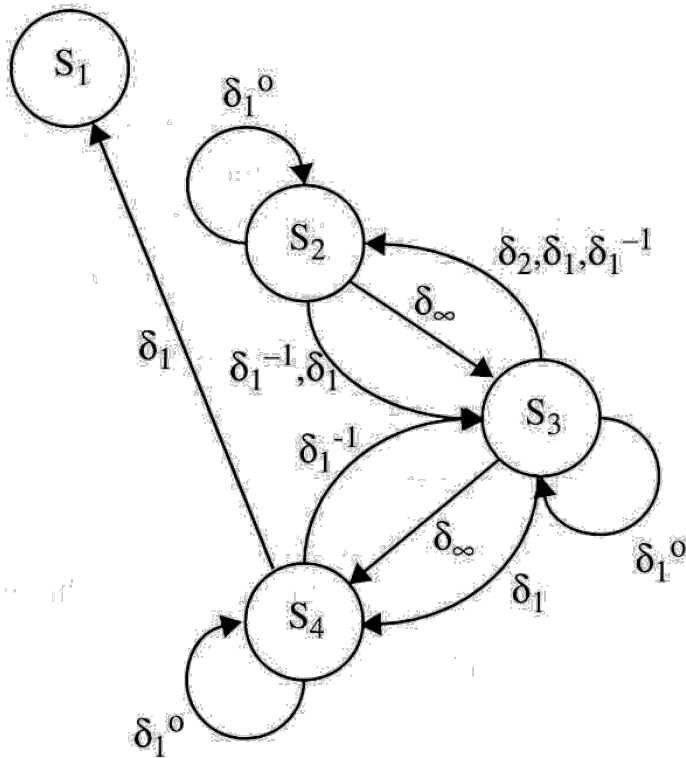
```
DO I = 1, 100
S1    X(I) = Y(I) + 10
      DO J = 1, 100
S2        B(J) = A(J,N)
          DO K = 1, 100
S3            A(J+1,K) = B(J) + C(J,K)
          ENDDO
S4    Y(I+J) = A(J+1, N)
      ENDDO
ENDDO
```



```
DO I = 1, 100
    codegen({S2, S3, S4}, 2))
ENDDO
Parallel DO I = 1, 100
S1: X(i) = Y(i) + 10
ENDDO
```

Example

- level-1 dependencies are removed
- codegen ($\{S_2, S_3, S_4\}, 2\}$)



Example

- S3 will be parallelized

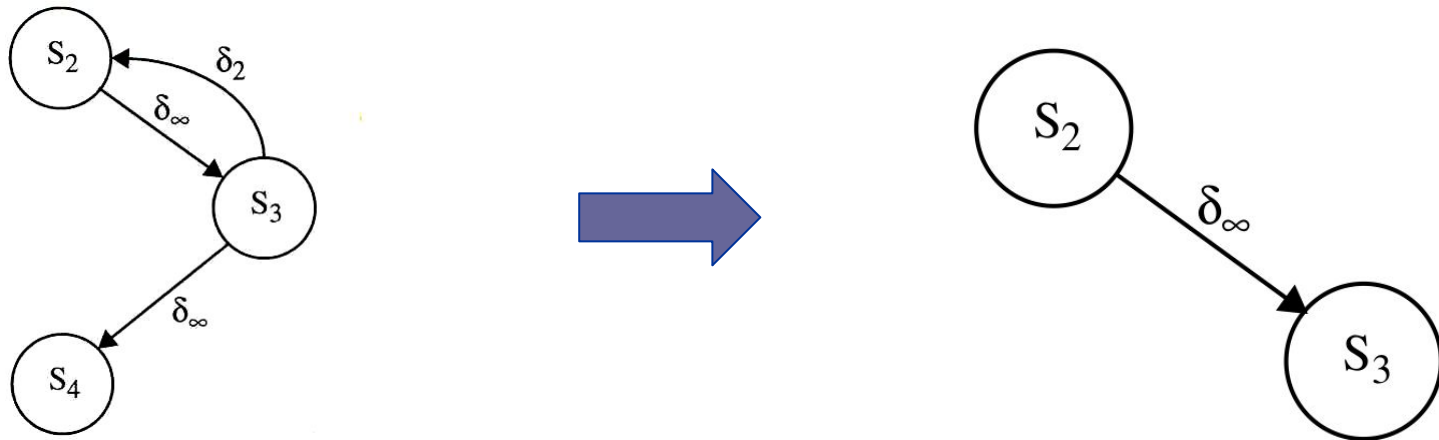
```
DO I = 1, 100
  codegen({S2, S3, S4}, 2})
ENDDO
Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```

```
DO I = 1, 100
  DO J = 1, 100
    codegen({S2, S3}, 3})
  ENDDO
Parallel DO J=1, 100
S4 Y(I+J) = A(J+1,N)
  ENDDO
ENDDO

Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```


Example

- level-2 dependencies are removed
- codegen ($\{S_2, S_3\}, 3\}$)



Example

- S3 will be parallelized

```
DO I = 1, 100
  DO J = 1, 100
    codegen({S2, S3}, 3))
  ENDDO
  Parallel DO J=1, 100
S4  Y(I+J) = A(J+1,N)
  ENDDO
ENDDO

Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```



```
DO I = 1, 100
  DO J = 1, 100
S2: B(J) = A(J,N)
    Parallel DO K = 1, 100
S3      A(J+1,K)=B(J)+C(J,K)
    ENDDO
  ENDDO
  Parallel DO J=1, 100
S4  Y(I+J) = A(J+1,N)
  ENDDO
ENDDO

Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```

Exercise

- S3 will be parallelized

```
DO I = 1, 100
  DO J = 1, 100
    codegen({S2, S3}, 3))
  ENDDO
  Parallel Do J=1, 100
S4 Y(I+J) = A(J+1,N)
  ENDDO
ENDDO

Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```



```
DO I = 1, 100
  DO J = 1, 100
S2: B(J) = A(J,N)
    Parallel DO K = 1, 100
S3      A(J+1,K)=B(J)+C(J,K)
    ENDDO
  ENDDO
  Parallel Do J=1, 100
S4 Y(I+J) = A(J+1,N)
  ENDDO
ENDDO

Parallel DO I = 1, 100
S1: X(I) = Y(I) + 10
ENDDO
```

Exercise

```
for (int i=10; i<100; i++) {  
    for (int j=10; j<100; j++) {  
        for (k=10; k<100; k++) {  
S1:            A[i][j][k] = B[i][j-10][k] + 10;  
S2:            C[i-2][j-2][k-3] = A[i-1][j][k] + E[i][j-5][k];  
S3:            B[i][j][k] = C[i-2][j-2][k-3] + D[i][j][k];  
S4:            F[i][j][k] = B[i-2][j][k] + D[i][j][k];  
                }  
        }  
    }
```

- Construct the dependency matrix/table
- Construct dependency graph
- Extract parallelism

References

- Slides of the book: Randy Allen, Ken Kennedy, Optimizing Compilers for Modern Architectures: A Dependence-based Approach, Morgan Kaufmann, 2001

Questions?

Thank you!